CST1510 Programming for data communication and networks

Week 7: OOP implementation

LAB ACTIVITY

ACTIVITY:

Lab Activity: Refactor Your Project into Object-Oriented Programming (OOP)

This week, you will extend your chosen project—either the **Smart Vending Machine** or the **Crypto Investment Platform**—by refactoring it into an Object-Oriented Programming (OOP) structure. This activity will help you organize your code, make it more modular, and prepare you for implementing advanced features in future weeks.

Instructions

1. **Review Your Text-Based Implementation**:
   - Revisit the text-based version of your chosen project (either the Smart Vending Machine or Crypto Investment Platform) that you developed in the previous lab.
   - Identify areas of your code that could be improved by using classes and methods.
2. **Plan Your OOP Structure**:
   - Think about how to divide your project into classes. Some suggestions:
     - For the **Smart Vending Machine**:
       - **Product**: A class to represent each digital product, with attributes like `id`, `name`, `price`, and `stock`.
       - **Cart**: A class to manage items added to the cart, with methods for adding, removing, and displaying items.
       - **VendingMachine**: A class to handle product inventory, load and save data, and process transactions.
     - For the **Crypto Investment Platform**:
       - **Account**: A class to handle user account details, including `username`, `balance`, and methods for depositing and withdrawing funds.

    - **Asset**: A class to represent each asset (cryptocurrency or precious metal), with attributes like `name`, `price`, and `quantity`.
    - **Portfolio**: A class to manage all the user's assets, with methods for buying, selling, and displaying holdings.

3. **Refactor Your Code into OOP**:
   - **Create Classes**:
     - Define classes based on your project's requirements (e.g., `Product`, `Cart`, `VendingMachine` for the Smart Vending Machine, or `Account`, `Asset`, `Portfolio` for the Crypto Investment Platform).
   - **Move Functions into Methods**:
     - Transform your existing functions into methods within the appropriate classes.
     - For example, move the `add_to_cart()` function into the `Cart` class and convert it into a method.
   - **Initialize Objects**:
     - Use object instances in your main program to interact with the system.
     - For example, create an instance of `VendingMachine` to handle all vending machine operations.
4. **Test Your OOP Implementation**:
   - Run your code to ensure that the refactored version works as expected.
   - Verify that you can still perform all essential operations, such as viewing products, adding items to the cart, and checking out, using your new class-based structure.
5. **Document Your Changes**:
   - Write a brief summary of the changes you made when refactoring to OOP.
   - Explain how the use of classes and objects improved the organization and readability of your code.
   - Reflect on any challenges you faced during the refactoring process and how you overcame them.

Example OOP Structure for the Smart Vending Machine

```python
# Example Product Class
class Product:
    def __init__(self, id, name, price, stock):
        self.id = id
        self.name = name
        self.price = price
        self.stock = stock

    def update_stock(self, quantity):
        if self.stock >= quantity:
            self.stock -= quantity
            return True
        return False
```

```python
# Example Cart Class
class Cart:
    def __init__(self):
        self.items = {}

    def add_item(self, product, quantity):
        if product.id in self.items:
            self.items[product.id]['quantity'] += quantity
        else:
            self.items[product.id] = {'name': product.name,
'price': product.price, 'quantity': quantity}

    def view_items(self):
        # Display cart items
        pass

# Example VendingMachine Class
class VendingMachine:
    def __init__(self, inventory_file):
        self.inventory = self.load_inventory(inventory_file)

    def load_inventory(self, inventory_file):
        # Load inventory from file
        pass

    def display_products(self):
        # Display available products
        pass
```

OOP Refactoring for the Shopping System (Week 7) - that you can adapt for your coursework projects:

OOP Refactoring for the Shopping System

Objective: Refactor the existing text-based shopping system into an Object-Oriented Programming (OOP) structure, using classes and objects to improve code organization and functionality.

Step 1: Define Core Classes

1. **Product Class**

    - **Attributes**:
        - `id` : A unique identifier for each product.
        - `name` : The name of the product.
        - `price` : The cost of the product.
        - `stock` : The available quantity in the inventory.
    - **Methods**:

    - `update_stock(quantity)` : Adjusts the stock level when items are purchased or restocked.
    - `__str__()` : Returns a string representation of the product, showing the name, price, and available stock.
2. **Cart Class**
    - **Attributes**:
        - `items` : A dictionary to store products and their quantities.
    - **Methods**:
        - `add_item(product, quantity)` : Adds a product to the cart, checking if enough stock is available. If so, it reduces the stock from the product.
        - `view_items()` : Displays the current contents of the cart, showing each product's name, quantity, and total cost.
        - `calculate_total()` : Calculates the total cost of all items in the cart.
        - `clear_cart()` : Empties the cart after checkout.
3. **ShoppingSystem Class**
    - **Attributes**:
        - `inventory` : A list or dictionary of all available `Product` objects.
    - **Methods**:
        - `load_inventory()` : Loads product data from a file or database to initialize the inventory.
        - `display_products()` : Shows all available products with details like name, price, and stock.
        - `checkout(cart)` : Handles the checkout process, calculates the total, and saves the transaction.
        - `save_transaction(cart)` : Logs the transaction details to a file or database for future reference.

Step 2: Refactor Functions into Class Methods

1. **Refactor Inventory Loading**
    - Move the `load_inventory()` function into the `ShoppingSystem` class.
    - Ensure that products are instantiated as `Product` objects and added to the inventory.
2. **Refactor Product Display**
    - The `display_products()` method in the `ShoppingSystem` class should iterate through the `inventory` and print each product using the `__str__()` method from the `Product` class.
3. **Refactor Cart Management**
    - The `add_to_cart()` function should become a method of the `Cart` class, ensuring that stock checks and updates are managed within the method.

- The `view_cart()` function should become `view_items()` in the `Cart` class, providing a clear breakdown of items and their costs.

4. **Refactor Checkout Process**

   - The `checkout()` function should be a method of the `ShoppingSystem` class, which interacts with the `Cart` object to calculate the total and clear the cart after payment.
   - Ensure the `save_transaction()` method records the purchase details, such as products bought, quantities, and total amount.

Example Code Implementation

```
# Product Class
class Product:
    def __init__(self, id, name, price, stock):
        self.id = id
        self.name = name
        self.price = price
        self.stock = stock

    def update_stock(self, quantity):
        if self.stock >= quantity:
            self.stock -= quantity
            return True
        return False

    def __str__(self):
        return f"{self.name}: ${self.price} (Stock:
{self.stock})"

# Cart Class
class Cart:
    def __init__(self):
        self.items = {}

    def add_item(self, product, quantity):
        if product.update_stock(quantity):
            if product.name in self.items:
                self.items[product.name] += quantity
            else:
                self.items[product.name] = quantity
            print(f"Added {quantity} of {product.name} to cart.")
        else:
            print(f"Insufficient stock for {product.name}.")

    def view_items(self):
        print("Cart Contents:")
        total = 0
        for name, quantity in self.items.items():
            total += product.price * quantity
            print(f"{name}: {quantity} items")
        print(f"Total Cost: ${total:.2f}")
```

```python
    def calculate_total(self):
        return sum(product.price * quantity for product, quantity
in self.items.items())

    def clear_cart(self):
        self.items.clear()

# Shopping System Class
class ShoppingSystem:
    def __init__(self):
        self.inventory = []

    def load_inventory(self):
        # Example of loading products (could be from a file or
database)
        self.inventory.append(Product(1, "Python Programming
eBook", 10.99, 25))
        self.inventory.append(Product(2, "Photo Editing License",
29.99, 15))
        self.inventory.append(Product(3, "Study Playlist MP3",
5.99, 50))

    def display_products(self):
        for product in self.inventory:
            print(product)

    def checkout(self, cart):
        total = cart.calculate_total()
        print(f"Total amount to pay: ${total:.2f}")
        cart.clear_cart()
        self.save_transaction(cart)
        print("Checkout complete.")

    def save_transaction(self, cart):
        # Save transaction details to a file or database
        pass

# Main Program
if __name__ == "__main__":
    system = ShoppingSystem()
    system.load_inventory()
    system.display_products()

    cart = Cart()
    cart.add_item(system.inventory[0], 2)
    cart.view_items()
    system.checkout(cart)
```

Implementation for Your Coursework Projects

1. **Smart Vending Machine Project**:

    - Implement the `Product` class to represent digital goods.

- Use a `VendingMachine` class that handles inventory, cart management, and transactions.
- Refactor inventory management and purchase processes to be object-oriented.

2. **Crypto Investment Platform**:

   - Implement `Asset` and `Portfolio` classes to manage investments.
   - Create an `InvestmentPlatform` class for user account management, trades, and portfolio updates.
   - Ensure all trading logic and data management are encapsulated within appropriate classes.

This refactored approach will make your projects more modular, easier to maintain, and align well with OOP principles.

In [ ]: